COMP473: Formal Aspects of Concurrent Systems

Annie Luxton: 300046696

4 October 2003

Questions

Expand on the following 5 correctness safety properties mentioned in the paper entitled *"Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms"* by Maged M. Michael and Michael L. Scott.

1. The linked list is always connected.
2. Nodes are only inserted after the last node in the linked list.
3. Nodes are only deleted from the beginning of the linked list.
4. *Head* always points to the first node in the linked list.
5. *Tail* always points to a node in the linked list.

Answers

Expansion on property 1 - The linked list is always connected.

By (informal) induction:

* Show that 1. holds initially:

There is only one node, the dummy node, in the linked list initially. Therefore, the linked list is trivially connected.

* Show that 1. is not falsified by any enqueue operation execution:

Assume the linked list is connected. Then only the last node in the linked list has a NULL as it's next pointer. The enqueue operation only inserts new nodes after the last node in the linked list (property 2). Line E8 in the enqueue operation ensures that E9, the linearization point of the enqueue operation where a new node is actually inserted into the linked list, only occurs if the *tail* pointer points at the last node in the linked list. If *tail* is pointing at the last node in the linked list (which is checked by testing whether it's next pointer is equal to NULL), then E9 uses a CAS to set *tail*'s next pointer to point at the new node, which is initialized to have a NULL as it's next pointer. Therefore, any enqueue operation will maintain the property that the only node to have a NULL in it's next pointer (thus not connecting it to any nodes to the right of it) is the last node in the linked list. Therefore, the enqueue operation maintains 1.

* Show that 1. is not falsified by any dequeue operation execution:

Assume the linked list is connected. Then only the last node in the linked list has a NULL as it's next pointer. The dequeue operation only deletes nodes from the beginning of the linked list (property 3). Assuming that all pointers are pointing at the correct nodes, and that the CAS on line D13 succeeds, then the *Head* pointer will be set to point at *Head*'s next pointer, that is, the *Head* will no longer point at the dummy node, it will now point at the dummy node's next pointer. At this point, no nodes have been disconnected from the linked list since no node has be freed yet. The original dummy node that was *Head* still points at *Head*'s next pointer, but so does *Head* now. Now that the original dummy node is not needed anymore and all the above has been done, the old dummy node is freed, thus disconnecting it from the rest of the linked list. This is the only disconnecting that occurs during a dequeue operation. Therefore, the dequeue operation maintains 1.

Expansion on property 2 - Nodes are only inserted after the last node in the linked list.

By (informal) induction:

* Show that 2. holds initially:

Initially, there is only one node in the linked list: the dummy node which has a NULL as it's next pointer. Therefore, this is the 'last' and only node in the linked list initially. At the initialisation, *Tail* is set to point at this dummy node. Line E9 of the enqueue operation uses a CAS to insert the new node by setting the next pointer of the node pointed at by *Tail* to point at the new node, provided that this pointer originally pointed at NULL. Since at initialisation, the dummy node was originally pointed at by *Tail* and this node's next pointer was set to NULL, then the new node is inserted after the dummy and last node in the linked list. Therefore, because of the use of a dummy node, 2. is maintained at initialisation.

* Show that 2. is not falsified by any enqueue operation execution:

Assume that all nodes have been inserted after the last node in the linked list. Line E7 of the enqueue operation checks whether the *tail* pointer points at the same node that *Tail* points at. If this is true, then E8 checks whether the next pointer of *tail*, *next*, is NULL. Since the linked list is always connected (property 1) then the only next pointer that should be equal to NULL should be the next pointer of the very last node in the linked list. Therefore, if both E7 and E8 evaluate to true, together they tell us that *Tail*'s next pointer is NULL and that therefore if property 1. holds (which it does), *Tail* is indeed pointing at the last node in the linked list.

If E8 evaluates to false, that is, if *Tail*'s next pointer, *next* is NOT set to NULL, then we know by property 1. that *Tail* cannot be pointing at the last node in the linked list. Therefore, control jumps to line E13 of the enqueue operation which swings *Tail* to the next node in an attempt to bring it closer to the last node in the linked list. This is repeated by a loop until *Tail*'s next pointer, *next* is equal to NULL and so *Tail* must point at the last node in the linked list.

Only once we know that *Tail* is pointing at the last node in the linked list can the new node be inserted. At this point, line E9 inserts the new node into the linked list by means of a CAS that sets *tail*'s (which is equal to *Tail*) next pointer to point at the new node. Therefore, the new node is inserted after the last node in the linked list and property 2. is maintained.

* Show that 2. is not falsified by any dequeue operation execution:

The dequeue operation does not insert any nodes, therefore it cannot falsify 2. The dequeue operation therefore maintains 2.

Expansion on property 3 - Nodes are only deleted from the beginning of the linked list.

By (informal) induction:

* Show that 3. holds initially:

Initially, there is only one node in the linked list: the dummy node which has a NULL as it's next pointer. Therefore, this is the 'first' and only node in the linked list initially. At the initialisation, both *Tail* and *Head* are set to point at this dummy node. The dequeue operation checks whether these two are equal. If they are, then if *Head*'s next pointer is set to NULL, then we know that the linked list is empty. This causes the dequeue operation to return FALSE on line D8. Therefore, initially, no nodes can be deleted and therefore 3. is maintained.

* Show that 3. is not falsified by any enqueue operation execution:

The enqueue operation does not delete any nodes, therefore it cannot falsify 3. The enqueue operation therefore maintains 3.

* Show that 3. is not falsified by any dequeue operation execution:

Assume that all nodes have been deleted from the beginning of the linked list. The dequeue operation sets a pointer *head* to point at *Head*, a pointer *tail* to point at *Tail* and a pointer *next* to point at the node pointed at by *head*'s next node. Line D6 of the dequeue operation checks whether the *tail* and *head* pointers point at the same node. This is to ensure that no other operation has changed the *Tail* or *Head* pointers since the beginning of the dequeue operation. If this is true, then it can mean one of two things: either the queue is empty or the *Tail* pointer is lagging.

The case in which the queue is empty causes the dequeue operation to quit and return FALSE, therefore maintaining 3.

The case in which the *Tail* pointer is lagging causes the CAS at line D10 to swing the *Tail* pointer to it's next pointer. This is done to ensure that *Tail* always points at a node in the linked list (property 5). Only once we know the queue is not empty and that *Tail* does not point at the same point as *Head*, the first node in the linked list, which is always pointed at by *Head* (property 4), can be deleted by means of a CAS at line D13. D13 sets *Head* to point at the node pointed at by it's next pointer, that is, the second node in the linked list. The first node in the linked list can then be freed, deleting it from the linked list for ever. Therefore, nodes can only be deleted from the beginning of the linked list and property 3. is maintained.

Expansion on property 4 - *Head* always points to the first node in the linked list.

By (informal) induction:

* Show that 4. holds initially:

Initially, there is only one node in the linked list: the dummy node which has a NULL as it's next pointer. Therefore, this is the 'first' and only node in the linked list initially. At the initialisation, both *Head* and *Tail* are set to point at this dummy node. Therefore, *Head* points to the first node in the linked list initially and 4. is maintained.

* Show that 4. is not falsified by any enqueue operation execution:

Assume *Head* points to the first node in the linked list. An enqueue operation does not manipulate the *Head* pointer at all, therefore it cannot move it to any other node in the linked list. Enqueue operations do not delete any nodes, that is, do not 'free' any nodes from the linked list. Therefore, there is no chance that *Head* will be left pointing at NULL after an enqueue operation execution. Therefore, *Head* still points at the first node in the linked list and 4. is maintained.

* Show that 4. is not falsified by any dequeue operation execution:

Assume *Head* points to the first node in the linked list. The dequeue operation sets a pointer *head* to point at the same node that *Head* points at, that is, the first node in the linked list. The only line in the dequeue operation that manipulates which node *Head* points at is D13, the linearization point of the dequeue operation where a node is actually removed from the linked list.

If the linked list is empty, that is, has only one dummy node in it, then the test at D7 will ensure that the dequeue operation does not reach line D13 and instead returns FALSE, not actually dequeueing any nodes at all. This stops *Head* from ever pointing at NULL and ensures that *Head* always points at a node in the linked list.

If the linked list is not empty, that is contains the dummy node as well as one or more other nodes, then if the CAS at line D13 evaluates to true, *Head* is set to point at *Head*'s next pointer, that is, the second node in the linked list. Only once this has been done and before the end of the dequeue operation the original first node in the linked list is removed by 'freeing' it at line D19. At this point the first node in the linked list is what used to be the second and *Head* points to it already by line D13. Therefore, *Head* still points at the first node in the linked list and 4. is maintained.

Expansion on property 5 - *Tail* always points to a node in the linked list.

By (informal) induction:

* Show that 5. holds initially:

Initially, there is only one node in the linked list: the dummy node which has a NULL as it's next pointer. Therefore, this is the 'first' and only node in the linked list initially. At the initialisation, both *Tail* and *Head* are set to point at this dummy node. Therefore, *Tail* points to a node in the linked list initially and 5. is maintained.

* Show that 5. is not falsified by any enqueue operation execution:

Assume *Tail* points to a node in the linked list. After creating the new node, flow of control immediately enters a loop within the enqueue operation. The body of this loop begins by setting a pointer *tail* to point at the node that *Tail* points at and a pointer *next* to point at the *tail*'s next pointer. At this point, either *Tail* points at a node other than the last node in the linked list or it points to the last node in the linked list. Line E8 checks this by comparing the *next* pointer to NULL. If E8 evaluates to false it means that *Tail* is not pointing at the last node in the linked list. Alternatively, if E8 evaluates to true it means that *Tail* is indeed pointing at the last node in the linked list.

If line E8 evaluates to false, that is, if *Tail* does not point at the last node in the linked list, then control jumps to line E13 which swings *Tail* to the next node (note that *Tail* is never swung to point at the previous node, therefore it can never point at a node before *Head* in the linked list). Control is then returned to the top of the loop. Near the top of the loop body, the *next* pointer (*Tail*'s next pointer) is compared to NULL at line E8. Every time that line E8 evaluates to false, that is, if *Tail* is not pointing at the last node in the linked list, line E13 is again executed in order to swing *Tail* to the next node. The loop is repeated until line E8 evaluates to true, that is until the *next* pointer is found to be equal to NULL at which point we know that *Tail* is pointing at the last node in the linked list.

If line E8 evaluates to true, that is, if pointer next is set to NULL, we know that *Tail* must point at the last node in the linked list by property 1. In this case, line E9 attempts to insert a new node at the end of the linked list, but does not change the *Tail* pointer at this point. Once this insertion has been completed successfully, control jumps to line E17 which uses a CAS to swing *Tail* to the next node in the linked list. A CAS is used incase another concurrent enqueue or dequeue operation has already moved the *Tail* pointer to the next node, so as to avoid letting it ever point to NULL.

In both the above cases, care is taken to ensure that *Tail* is always pointing at a node in the linked list. Once it is pointing at the last node in the linked list, the new node can be added and then if *Tail* is still pointing at the second to last node in the linked list, an attempt is made to move *Tail* to point at the newly added last node. Therefore, *Tail* always points to a node in the linked list and 5. is maintained.

* Show that 5. is not falsified by any dequeue operation execution:

Assume *Tail* points to a node in the linked list. There are two important cases that can occur when dequeueing a node from the linked list. The queue can be empty, or the *Tail* pointer can be lagging and in particular be pointing at the first node in the linked list. We are not interested in the case that the queue is not empty and the *Tail* pointer is not lagging and is indeed pointing at the last node in the queue, or in the case that the queue is not empty and the *Tail* pointer is lagging but not far enough to be pointing at the first node in the linked list. The reason for this is that the dequeue operation only ever dequeues nodes from the beginning of the linked list (property 3.) and therefore in these cases, dequeue would not affect the nodes that *Tail* may be pointing at. Thus, meaning that in these two latter cases, *Tail* would not be touched and would remain pointing at a node in the linked list after a dequeue operation.

In the case that the queue is empty, that is, both *Head* and *Tail* pointers are pointing at the dummy node and *Head*'s next pointer is set to NULL, the dequeue operation will not dequeue any nodes and return FALSE at line D8. Therefore, *Tail* will remain pointing at a node in the linked list, the dummy node, maintaining 5.

The case in which the queue is not empty but the *Tail* pointer is lagging and points at the same node that *Head* points to (which by property 4. we know is the first node in the linked list) causes the CAS at line D10 to swing the *Tail* pointer to it's next pointer. This is done to ensure that when the first node in the linked list is removed (property 3.), *Tail* still points at a node in the list. Only once the *Tail* pointer has been moved away from pointing at the first node in the linked list and is instead set to point at the second is the first node in the linked list can a node be dequeued and freed. Therefore, *Tail* will still point at a node in the linked list, maintaining 5.